Assignment-13.4

Name: Arjun Manoj

H. No:2303A52134

Batch:44

Task 1: Refactoring Data Transformation Logic

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

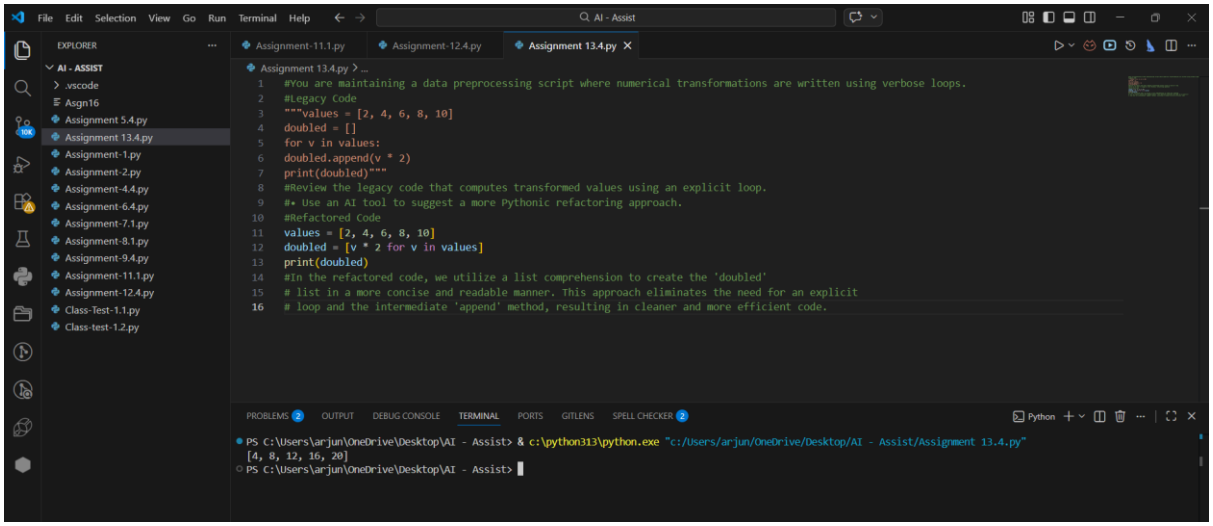
Prompt:

Refactor the following Python code into a more Pythonic form using list comprehensions while keeping the same output.

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)

print(doubled)
```

Code:



Observation:

The AI suggested replacing the explicit loop with a list comprehension, which makes the code shorter and more readable. The refactored code performs the same transformation but in a single line

Task 2: Improving Text Processing Code

Readability

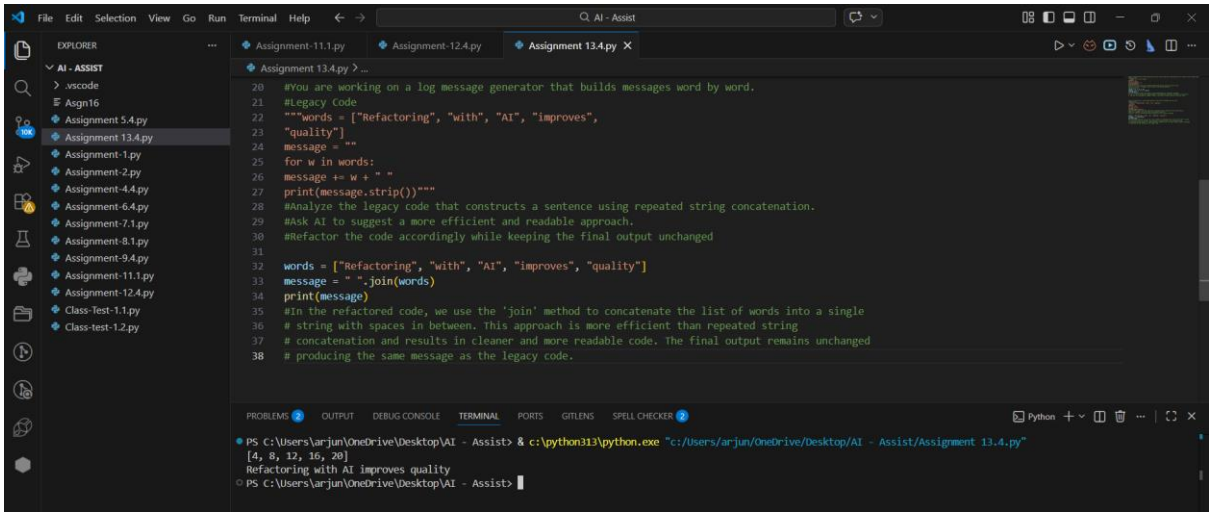
You are working on a log message generator that builds messages word by word.

Prompt:

Refactor the following Python code to make it more efficient and readable while keeping the output the same. Avoid repeated string concatenation.

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
message += w + " "
print(message.strip())
```

Code:



Observation:

The AI suggested using the join() method instead of repeatedly concatenating strings inside a loop.
Using " ".join(words) combines all words in the list into a single string separated by spaces. This approach is more efficient, faster, and cleaner than repeated string concatenation. The final output remains the same

Task 3: Safer Access to Configuration Data

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

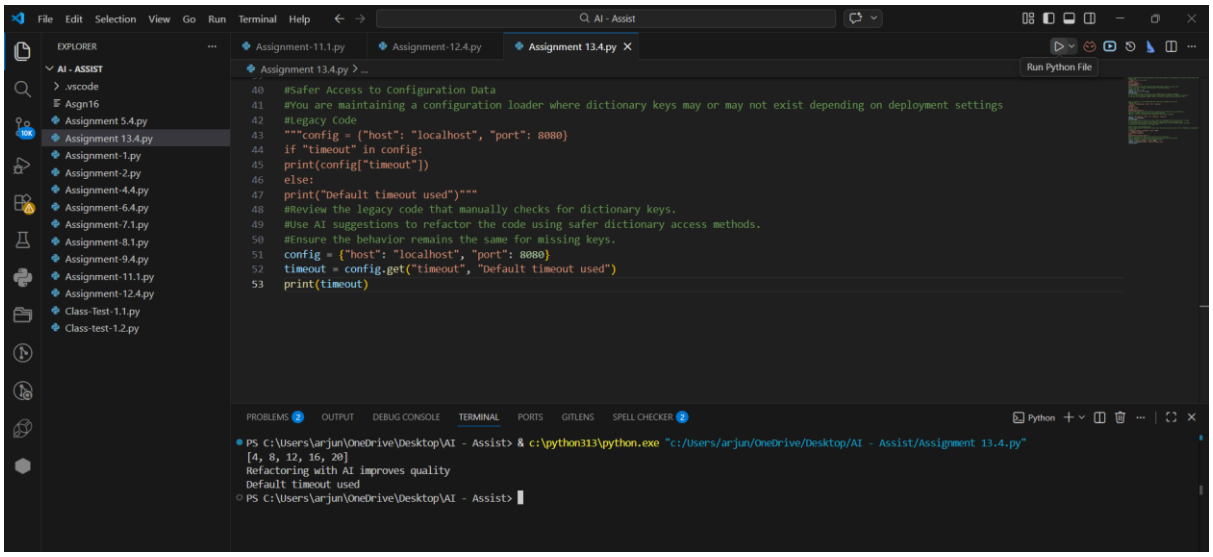
Prompt:

Refactor the following Python code to use a safer and cleaner way to access dictionary values when a key may not exist. The output behavior should remain the same.

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
print(config["timeout"])
```

else:
print("Default timeout used")

Code:



Observation:

The AI suggested using the get() method of dictionaries instead of manually checking if a key exists.
config.get("timeout", "Default timeout used") safely retrieves the value of "timeout" if it exists; otherwise it returns the default message. This makes the code shorter, cleaner, and safer while keeping the same behavior when the key is missing.

Task 4: Refactoring Conditional Logic for Scalability

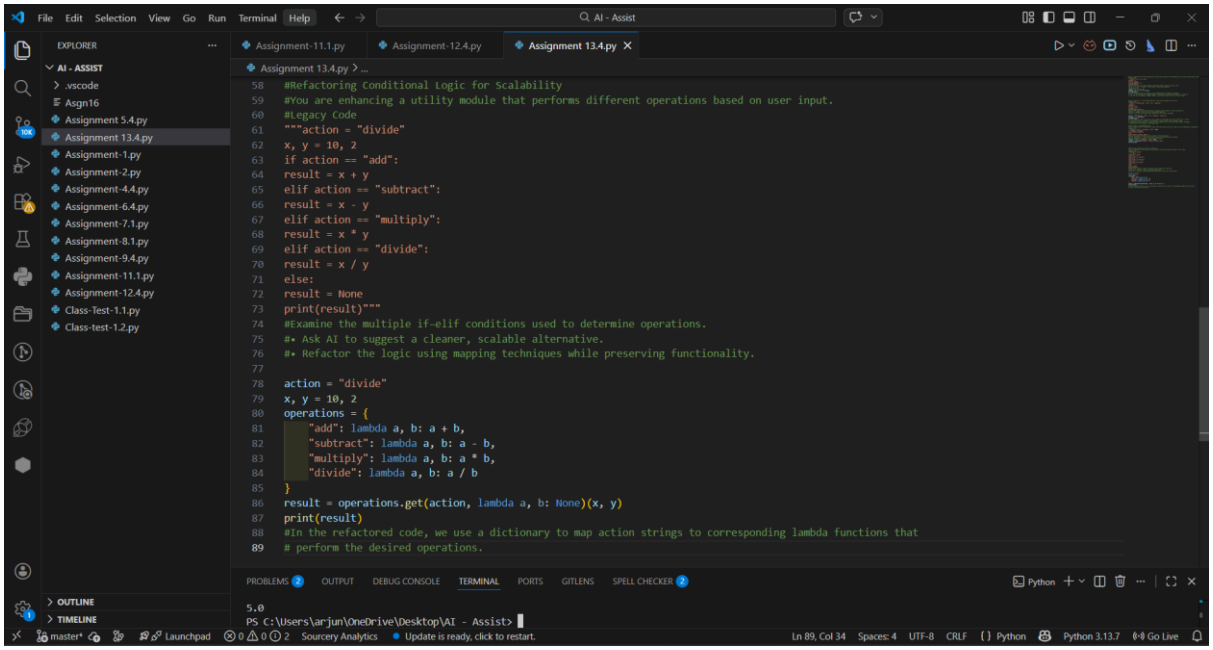
You are enhancing a utility module that performs different operations based on user input.

Prompt:

Refactor the following Python code to remove multiple if–elif conditions and use a cleaner, more scalable approach such as mapping techniques while keeping the same functionality.

```
action = "divide"
x, y = 10, 2
if action == "add":
result = x + y
elif action == "subtract":
result = x - y
elif action == "multiply":
result = x * y
elif action == "divide":
result = x / y
else:
result = None
print(result)
```

Code:



Observation:

The AI suggested replacing multiple if–elif conditions with a dictionary mapping technique.
Each operation name is mapped to a corresponding lambda function that performs the calculation. The get() method safely retrieves the required operation, and a default function returns None if the action is not found. This approach makes the code cleaner, easier to maintain, and more scalable when adding new operations.

Task 5: Simplifying Search Logic in Collections

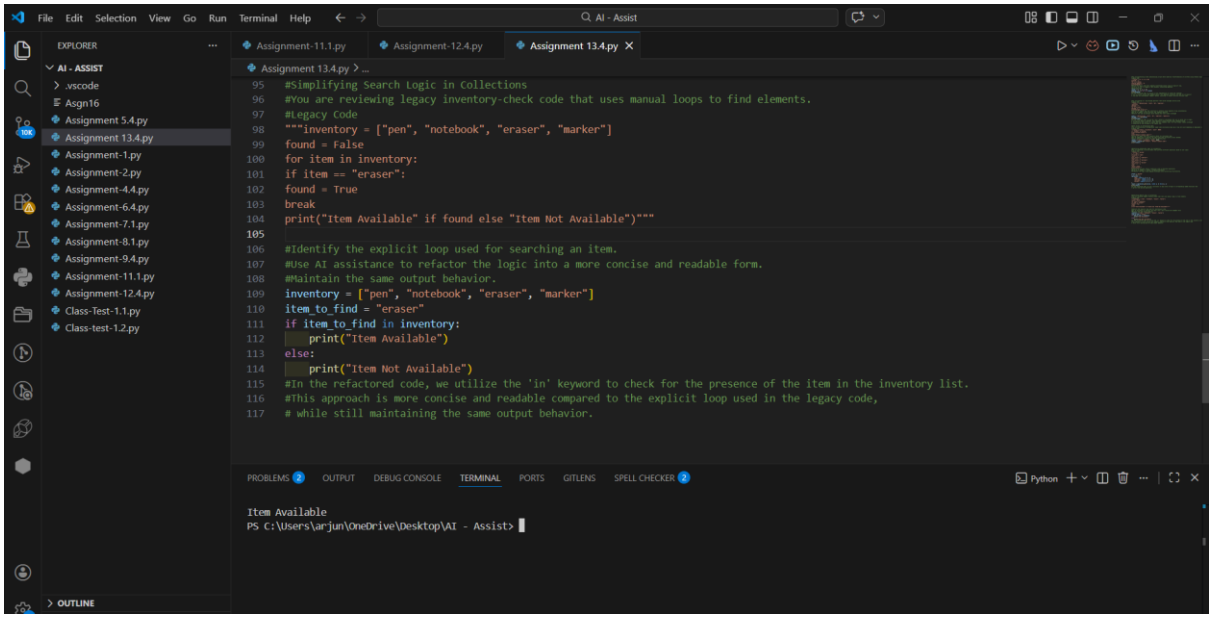
You are reviewing legacy inventory-check code that uses manual loops to find elements.

Prompt:

Refactor the following Python code to make the search logic more concise and readable. Avoid using an explicit loop if possible and maintain the same output behavior.

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
if item == "eraser":
found = True
break
print("Item Available" if found else "Item Not Available")
```

Code:



Observation:

The AI suggested replacing the explicit loop with the in membership operator. This operator directly checks whether an item exists in a list, eliminating the need for a manual loop and flag variable. The refactored code is shorter, more readable, and more Pythonic, while producing the same output as the legacy code.